Generation of Classical-Quantum Code from UML models

Ricardo Pérez-Castillo
Faculty of Social Science and IT
University of Castilla-La Mancha
Talavera de la Reina, Spain
ricardo.pdelcastillo@uclm.es

Luis Jiménez-Navajas
Faculty of Social Science and IT
University of Castilla-La Mancha
Talavera de la Reina, Spain
luis.jimeneznavajas@uclm.es

Iván Cantalejo
Faculty of Social Science and IT
University of Castilla-La Mancha
Talavera de la Reina, Spain
ivan.cantalejo@alu.uclm.es

Mario Piattini
Escuela Superior de Informática
University of Castilla-La Mancha
Ciudad Real, Spain
mario.piattini@uclm.es

2023 IEEE International Conference on Quantum Computing and Engineering (QCE) | 979-8-3503-4323-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/QCE57702.2023.10202

***Abstract*— Quantum computing has gained a lot of interest in recent years, thanks to its ability to solve certain problems that classical computing cannot address. These applications will only be achieved through the development of quantum software. Thus, many organizations are starting projects to integrate this new software into their existing systems. Software modernization (as based on Model-Driven Engineering) can be used to migrate from/to the so-called hybrid software systems, which integrate classical and quantum software. In that process, reverse engineering and restructuring phases have already been addressed, but the forward engineering phase for hybrid software has not yet been addressed. This paper presents a preliminary code generation technique from extended UML design models. It proposes an EGL transformation to generate Python and Qiskit code by integrating classical and quantum code. The transformation has been preliminarily validated with an example of application with the design model of a hybrid system for a financial web application. The implication of this work is that the software modernization process for hybrid software systems is completed, which facilitates the adoption of this new paradigm in the industry.**

Keywords—quantum software, code generation, MDE, UML, EGL

I. Introduction

Quantum Computing (QC) is set to revolutionize the way certain problems will be solved which, until now, pose a computational challenge for classical computers. That is, those problems that cannot be addressed in a reasonable time frame, even using the world's largest classical supercomputer.

In the last few years, the interest in QC has been dramatically increased. The present decade (2020s) is known as the "quantum decade", in which QC "is poised to expand the scope and complexity of the business problems we can solve" [1], offering a true "quantum advantage". Thus, QC is gaining more and more relevance due to its potential applications in various fields and business sectors [1, 2]. QC is, therefore, at an inflection point with significant barriers to cross yet a world of opportunities ahead [3]. This means that companies must be prepared for this new technological transition.

Quantum supremacy has already been demonstrated or, at least, claimed [4], and more powerful quantum computers are progressively appearing, e.g., Google has predicted a 1 million qubits machine before the end of the decade [5]. Despite the preliminary demonstration of many of these cutting-edge advances and their promising applications, the advantages offered by QC cannot be exclusively achieved by using quantum computers in isolation, but also requires quantum software, which will undoubtedly play an important role [6].

The existing quantum software programming techniques have been experimentally proposed, in most of the cases, in an *ad hoc* way. Consequently, Quantum Software Engineering (QSE) [7-9] has emerged as a new research field aimed at producing quantum software by applying the knowledge and lessons learned from the field of classical software engineering. This requires the application or adaptation of existing software engineering principles, processes, methods, techniques, and practices for quantum software development, as well as the proposal of new ones.

Among other, QSE is aimed to address an important challenge: how to migrate from/to the so-called hybrid software systems [10]. This is a challenge as companies will not fully discard their classic software systems. Classical systems could still support critical business processes. Even more, using quantum software for some simplistic functionalities might not make sense. Rather than a full replacement, classical and quantum information systems will operate together in the so-called hybrid software systems [10]. These systems support business functionality implemented with classical software (*driver*) that make calls to quantum software, typically running in remote quantum computers.

A possible strategy for migration from/to hybrid software systems is software modernization. That process is based on traditional reengineering by follows the Model-Driven Engineering (MDE) principles. Software modernization consists of three phases: (i) reverse engineering, (ii) restructuring; and (iii) forward engineering. So far, there are some proposals that have already addressed the reverse engineering phase [11], as well as the restructuring phase [12]. However, code generation from the high-level models, in the forward engineering phase, has not been addressed yet.

Of course, there are a lot of proposals, especially tools, that can generate quantum code from different quantum circuits representations (e.g., IBM Quantum Composer, Quantum Programming Study Modeler, among other). Nevertheless, the main contribution of this proposal is the generation of classical and quantum code in a combined way from design, high-level models as UML. In this way, relationships between classical and quantum parts can be used as relevant information to the target hybrid software system.

979-8-3503-4323-6/23/$31.00 ©2023 IEEE
DOI 10.1109/QCE57702.2023.10202

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:09:13 UTC from IEEE Xplore. Restrictions apply.

This research proposes a generative technique based on a *model-to-text* transformation that is able to generate Python and Qiskit [13] code from extended UML models (which consider high-level design of classical-quantum software). The transformation has been implemented in EGL (*Epsilon Generation Language*) and integrated into a supporting tool. This implementation has allowed a preliminary validation with a UML design of a small financial system that considers some quantum functionalities. The main implication of this work is that it covers the forward engineering phase, which allows to complete the whole software modernization process for hybrid software systems. Thus, it can facilitate the adoption of quantum software in the industry.

The remaining of this paper is organized as follows. Section II explains the software modernization process in which the proposal is framed in. Section III describes the technique to generate code from extended UML models. Section IV briefly presents an example of application. Finally, Section 0 discusses conclusions and future work.

II. Software Modernization Process

Hybrid software systems face obstacles in code portability, tool integration, program validation and verification, and workflow orchestration [14]. Another important challenge is the modernization toward hybrid software systems by following the existing good practices of the software engineering field, such as automation, ease of comprehension, abstraction of high-level designs [12], preservation of the embedded business knowledge [11], quality assurance [15], service-oriented software [16], among others. In this sense, software modernization has been adapted to address the problems associated with migrations of these hybrid software systems [10].

The software modernization process is based on traditional reengineering by adding the MDE principles, i.e., every software artefact is modeled at different abstraction levels, and those models are automatically transformed in other models. Software modernization consists of three phases: (i) reverse engineering, to inspect and abstract information at a higher level of abstraction; (ii) restructuring, to improve the internal design and add new functionalities at the highest level of abstraction; and (iii) direct engineering, to generate the target source code. In addition, this research considers a software modernization process based on existing standards such as UML (*Unified Modelling Language*) and KDM (*Knowledge Discovery Metamodel*) (see Fig. 1).

Hybrid software modernization can be used in at least three non-exclusive scenarios: (i) migrate legacy classical information systems to hybrid architectures that support classical-quantum software integration; (ii) migrate existing isolated quantum algorithms and integrate them into hybrid systems; and (iii) transform or add new business functionalities supported by quantum software, which will be integrated into the resulting hybrid systems. Any of these scenarios require, in the last phase, the code generation in a (semi)automatic way from abstract design models. It is precisely what is addressed by this work (see the highlighted part in Fig. 1).

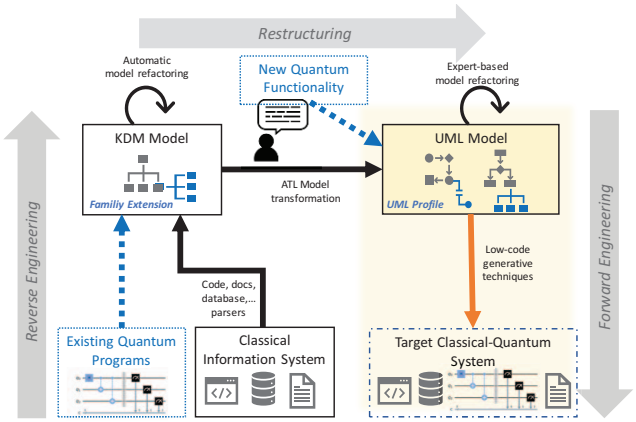

Fig. 1. Software Modernization Process

A. UML Designs for Hybrid Software Systems

In a previous work, a UML profile for classical-quantum software was proposed [17]. This profile fulfilled the need to design and build quantum software systematically [18]. Abstract software descriptions are key for (i) discussing design concerns between developers, and (ii) modeling systems with high-level conceptual representations, while low-level implementation details remain hidden. Due to this, UML [19] was chosen in this research since it is a well-proven solution that is widely used in the classical software industry. The proposed UML profile [17] addresses use case, class, sequence, activity, and deployment diagrams. As a result, both quantum and classical elements, as well as the relationships between them, can be modeled together in an integrated design model, which is key in the software modernization process. There are other proposals for modelling quantum software with UML. For example, in [20] is defined some principles to be preserved when designing modelling languages for quantum software by providing a set of extensions to UML. In this sense, Ali and Yue [21] discusses some ideas for deriving new metamodels for modelling quantum programs as extensions of UML. Alternatively, other authors considered Domain-Specific Modelling Languages (DSML) as discussed in [22]. Some examples have been proposed for Quantum Machine Learning [23], as well as for modeling quantum circuits from satisfiability problems [24]. The expressiveness of DSML is usually higher. However, the main advantage of the UML profile (the standard extensibility mechanism), is the compliance and interoperability with existing techniques and tools.

B. Quantum Code generation tools

There are several tools for generating quantum source code from different quantum circuits models, most of them from graphical editors. For example, the IBM Quantum Composer [25] generates OpenQASM 2.0 and Qiskit code. Similar features are supported by the QI Editor in Quantum Inspire [26]. Quantum Programming Studio [27] has a quantum circuit modeler which supports the code generation of a large list of quantum programming languages. Similarly, QuAntiL circuit transformer [28] enables the translation of quantum circuits into different languages. However, none of those editors considers classical and quantum software in combination as this research proposes.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:09:13 UTC from IEEE Xplore. Restrictions apply.

III. Hybrid Code Generation from UML models

The proposed model-to-text transformation considers two types of UML diagrams according to the Quantum UML profile: (i) the class diagram which provides the comprehensive static view of the system, representing how classes and packages are organized; and (ii) the activity diagram which serves to model the dynamics (algorithms or quantum circuits) of certain quantum software components. The output for the target hybrid software is Python code and its quantum software extension, Qiskit [13]. Today, both languages are widely used, both in classical and quantum software development. In addition, Qiskit supports the generation of OpenQASM quantum assembly code, which is the basis of many other quantum programming languages. Despite this, the transformation could be adapted to other models and programming languages according to the MDE approach.

The transformation process is made in two steps, which can be considered as two independent model-to-text transformation executed in a row (the transformation code is available in [29]). In a first step, the quantum software is generated from the extended UML activity diagrams that represents quantum circuits. In a second step, the Python code (classical software) is generated from the UML class diagrams, which also uses quantum stereotypes defined in the UML profile. Such information is extremely relevant to define dependencies and relationships between classical and quantum code. Because of this, the quantum code generation is made first.

A. UML Activity Diagrams to Qiskit Code

The first EGL transformation generates a Python (*.py*) file for each activity diagram (that represent a quantum circuit). The Python file first add all the Qiskit imports and then it will generate a *QuantumCircuit* object representing the quantum circuit. All the quantum elements (registers, gates, measures, etc.) will be added to that object. First, a quantum register (by using *QuantumRegister* class) is generated, which groups the number of qubits that are used by the circuit. According to the Quantum UML profile, every *Lane* element represent a qubit in the circuit. Similarly, *a ClassicalRegister* object is generated (and added to the circuit) for every qubit affected by a measure operation (i.e., an UML *Action* element stereotyped with <<Meassure>>). Second, the *QuantumCircuit* object is instantiated by using as parameters both registers (quantum and classical). After that, quantum gates are added to the circuit, as well as the measure operations. Quantum gates are modelled in the UML activity diagrams as 'Action' elements stereotyped with <<QuantumGate>> (see Fig. 2). It is crucial to preserve the order of quantum gates, since in somehow quantum circuit does not define an explicit execution flow. Another important question is the controlled gates, which are modeled as a pair of two elements and one constraint between them: a *SendSignalAction* (on the qubit used to the control) and an *AcceptEventAction* (on the qubit in which the gate is applied). In this case, the EGL transformation should consider the relationship between those two elements applied in different *Lane* elements, at the same time the order of the remaining gates is kept. Finally, the circuit is generated and persisted in the target Python file through the EGL template.

```
1  [*QUANTUM GATES*]
2      [% var initial = getInitialNode();
3          if(initial == null){
4              "There is no InitialNode".println();
5          } else{
6              var actual = initial.outgoings.target.get(0);
7              var type = actual.toString().split(' ').get(0).split('@').get(0);
8              while(not(type == "org.eclipse.uml2.uml.internal.impl.ActivityFinalNodeImpl")){
9                  var gate = checkGateType(actual);
10                 if (not (gate == "pass")){%]
11                     [%=circuitName%].[%=gate%]
12                 [%]
13                 actual = nextNode(actual);
14                 type = actual.toString().split(' ').get(0).split('@').get(0);
15             }
16         }%]
```

Fig. 2. Excerpt of EGL transformation for quantum gates generation in Qiskit

B. UML Class Diagrams to Python Code

The second EGL transformation generates the Python code for the classical software, from the UML class diagrams. Of course, there already exist tools for classical code generation from UML class diagrams. The difference is that the classical code is generated with some relationships with the previous Qiskit code. In this sense, the first mapping considers *Packages* and generates the respective directory structure to store the Python files for each *Class*. For each *Class*, all the a*ttributes* are generated in code. It is necessary to establish a mapping between the predefined UML data types and the available Python data types (Of course this is parameterizable). Along with the *attributes*, the respective functions for the public interface, i.e., *'getters'* and *'setters'*, are defined. Finally, the *operations* defined in the UML *Classes* are defined as functions in the Python code. The function signature is created in the target class with the parameters and the return type. Obviously, these operations are empty and must be completed by developers.

Apart from those transformation rules, the novelty of this proposal is that the quantum code, generated previously, is integrated with the classical code. The following conditions are checked to do such integration: (i) There is a UML *class* stereotyped as <<Quantum Driver>>; (ii) There is a dependency from the previous *class* to a different UML *class* stereotyped as <<Quantum>> (which represents the quantum circuit); and (iii) the <<Quantum>> class name match with the name of the *.py* file previously generated from a UML activity diagram. As a result, the generated source code for the <<Quantum Driver>> class contains a Qiskit *QuantumCircuit* attribute with the name of the target <<Quantum>> class (see Fig. 3).

```
1  [%function getAttributeType(attribute) : String{
2      if(attribute.type.name == null)
3          return getDataType(attribute.type.eProxyURI.toString().split('#').get(1));
4      else if(attribute.type.name == "qiskit.circuit.QuantumCircuit")
5          for(activity in activities.replace(' ', '').split(','))
6              if(not(attribute.name == null))
7                  if((activity.replace(' ', '_').replace('-', '_')) ==
8                  (attribute.name.replace(' ', '_').replace('-', '_').toLowerCase()))
9                      return attribute.name;
10     return getDataType(attribute.type.name);
11 }
```

Fig. 3. Excerpt for the EGL function for classical-quantum code integration

IV. Application Example

EGL transformations can directly be executed from Papyrus (Eclipse). However, a JavaFX tool has been developed to execute the transformations for several UML diagrams representing a whole design for a hybrid software system. The proposal has been preliminary validated with a hybrid software system previously developed in a manual way. The system is a web application inspired in an example provided in the QisKit finance library for estimate they payoff in pricing European call options, which include some classical software functionalities [30]. In [17] the UML design for that system is provides, which

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:09:13 UTC from IEEE Xplore. Restrictions apply.

is used in this example as the input for generating Python/Qiskit code. The generated code and the UML models can be found together the EGL transformations in the same repository [29]. As a result, the generated code defines only the skeleton of the classic classes, while the existing code presents fully functional code. The generated code of the quantum circuit varies in the order of application of some gates, but only in cases where it does not affect the result, so those can be considered as functionally equivalent circuits. Other minor differences regarding the code manually generated was due to some changes that developers manually introduced regarding the UML design (which usually occurs in real development projects). However, this cannot be considered an error in the proposed transformation. The example of application preliminarily demonstrates the feasibility and suitability to generate fully functional Python/Qiskit code for hybrid software systems.

V. Conclusions and Future Work

Despite there are several generative techniques for both classical and quantum software, those were provided in isolation. Hence, the main contribution of this research is the combined generation of classical (Python) and quantum (Qiskit) code from the high-level design in UML of hybrid software systems. The proposed generative technique is preliminary and poses certain limitations, which are proposed as potential improvements to be addressed in future work. For example, to consider other elements in Python and Qiskit, such as predefined quantum software libraries, as well as to explore generation in other programming languages. The EGL transformation needs to validate in more case studies with multiple UML models of hybrid software systems and validate how the technique really works. This proposal contributes to completing the software modernization process from/to hybrid software systems. In this way, this proposal can boost the adoption of quantum software by companies, while helps them to migrate their existing classical-quantum systems by preserving the critical business knowledge embedded in the software. This adoption, in turn, will accelerate the transition to quantum computing/technology to realize the benefits of many of its promising applications.

Acknowledgment

This work has been supported by PID2019-104791RB-I00 (SMOQUIN) and PDC2022-133051-I00 (QU-ASAP) funded by MCIN/AEI/ and by the European Union/PRTR. This work is also part of the project 'QHealth: Quantum Pharmacogenomics Applied to Aging', funded by the 2020 CDTI Missions Program.

References

[1] IBM, *The Quantum Decade. A playbook for achieving awareness, readiness, and advantage*. 3rd ed. 2022: IBM Institute for Business Value.

[2] Ukpabi, D., et al., *Framework for understanding quantum computing use cases from a multidisciplinary perspective and future research directions.* arXiv, 2023. 2212.13909v2: p. 13.

[3] Lenahan, B., *Quantum Boost: Using Quantum Computing to Supercharge Your Business*. 2021: Library and Archives Canada.

[4] Arute, F., et al., *Quantum supremacy using a programmable superconducting processor.* Nature, 2019. 574(7779): p. 505-510.

[5] Livemint. *Google says it will build a commercial quantum computer before 2030*. 2021 19 May 2021 [cited 2021; https://www.livemint.com/technology/tech-news/google-says-it-will-build-a-commercial-quantum-computer-before-2030-11621363226405.html.

[6] Mueck, L., *Quantum software.* Nature, 2017. 549(7671): p. 171-171.

[7] Serrano, M.A., R. Pérez-Castillo, and M. Piattini, eds. *Quantum Software Engineering*. 2022, Springer. XXVIII, 302.

[8] Zhao, J., *Quantum Software Engineering: Landscapes and Horizons.* arXiv preprint, 2020.

[9] Carleton, A.D., et al., *Architecting the Future of Software Engineering: A National Agenda for Software Engineering Research and Development*. 2021, Software Engineering Institute, Carnegie Mellon University.

[10] Pérez-Castillo, R., M.A. Serrano, and M. Piattini, *Software modernization to embrace quantum technology.* Advances in Engineering Software, 2021. 151: p. 102933.

[11] Pérez-Castillo, R., L. Jiménez-Navajas, and M. Piattini, *QRev: migrating quantum code towards hybrid information systems.* Software Qual J, 2021.

[12] Pérez-Castillo, R., L. Jiménez-Navajas, and M. Piattini, *Modelling Quantum Circuits with UML*, in *43rd ACM/IEEE International Conference on Software Engineering Workshops. 2021 IEEE/ACM 2nd International Workshop on Quantum Software Engineering (Q-SE)*. 2021, IEEE Computer Society: Virtual (originally in Madrid, Spain). p. 7-12.

[13] Qiskit, I. *Getting Started with Qiskit*. 2022. https://qiskit.org/documentation/tutorials/circuits/1_getting_started_with_qiskit.html.

[14] McCaskey, A., et al., *Hybrid Programming for Near-Term Quantum Computing Systems*, in *2018 IEEE International Conference on Rebooting Computing (ICRC)*. 2018. p. 1-12.

[15] Cruz-Lemus, J.A., L.A. Marcelo, and M. Piattini, *Towards a Set of Metrics for Quantum Circuits Understandability*. 2021, Springer International Publishing: Cham. p. 239-249.

[16] Garcia-Alonso, J., et al., *Quantum Software as a Service Through a Quantum API Gateway.* IEEE Internet Computing, 2022. 26(1): p. 34-41.

[17] Pérez-Castillo, R. and M. Piattini, *Design of classical-quantum systems with UML.* Computing, 2022.

[18] Dey, N., et al. *The Quantum Development Life Cycle*. 2020 [08/01/2021]; https://arxiv.org/abs/2010.08053v1.

[19] OMG, *UML 2.5.1. https://www.omg.org/spec/UML/2.5.1/PDF*. 2017.

[20] Perez-Delgado, C.A. and H.G. Perez-Gonzalez, *Towards a Quantum Software Modeling Language*, *2020. IEEE/ACM 42nd International Conference on Software Engineering Workshops (ICSEW)*. 2020: Han River, Seoul, South Korea. p. 441-444.

[21] Ali, S. and T. Yue, *Modeling Quantum programs: challenges, initial results, and research directions*, in *Proceedings of the 1st ACM SIGSOFT International Workshop on Architectures and Paradigms for Engineering Quantum Software*. 2020, ACM: Virtual, USA. p. 14–21.

[22] Gemeinhardt, F., A. Garmendia, and M. Wimmer, *Towards Model-Driven Quantum Software Engineering*, *2021. IEEE/ACM 2nd Int. Workshop on Quantum Software Engineering (Q-SE)*. 2021. p. 13-15.

[23] Moin, A., et al., *Towards Model-Driven Engineering for Quantum AI*, in *INFORMATIK 2022*, D. Demmler, Krupka, D. & Federrath, H., Editor. 2022: Gesellschaft für Informatik, Bonn. p. 1121-1131.

[24] Alonso, D., P. Sánchez, and F. Sánchez-Rubio, *Engineering the development of quantum programs: Application to the Boolean satisfiability problem.* ADV ENG SOFTW. 2022. 173: p. 103216.

[25] IBM. *IBM Quantum Composer*. 2023; https://quantum-computing.ibm.com/composer.

[26] Last, T., et al. *Quantum Inspire: QuTech's platform for co-development and collaboration in quantum computing. Novel patterning technologies for semiconductors, MEMS/NEMS and MOEMS 2020*. 2020. SPIE.

[27] Quantastica. *Quantum Programming Studio*. 2019 [cited 2020; https://quantum-circuit.com/.

[28] Universy of Stuttgart. *Quantum Application Lifecycle Management (QuAntiL)*. 2023: https://quantil.readthedocs.io/en/latest/user-guide/circuit-transformer/.

[29] Cantalejo, I. *EGL scripts for transforming Quantum UML models into Python code*. 2023. https://github.com/ivyncm/PythonGenerator/tree/main/EGLtemplates.

[30] Alarcos, R.G. *Pricing European Call Options App*. 2022. http://alarcosj.esi.uclm.es:8080/.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:09:13 UTC from IEEE Xplore. Restrictions apply.